WBSとの動的協調による テーブルマッチング処理の高速化

名古屋工業大学大学院
〇渡辺 晃佑　川島 龍太　松尾 啓志

本発表の概要

目的

CNFにおけるテーブルマッチング処理の高速化

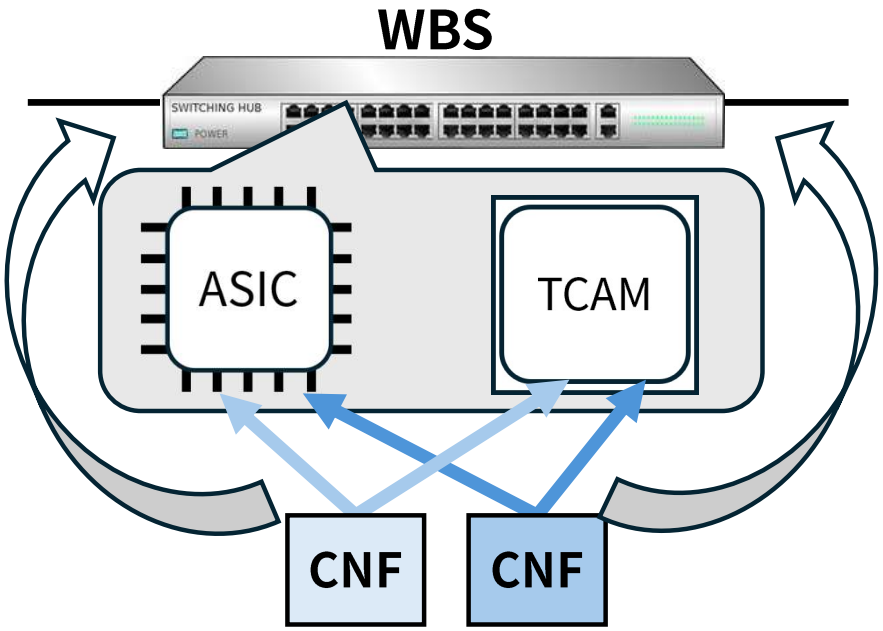
アイデア

CNF主体によるWBSとの動的な協調アーキテクチャ

結果

1024エントリで処理性能が最大7倍の高速化

負荷の高い処理
のオフロード



背景

◻ CNF (Cloud-native Network Function)

メリット
迅速なデプロイ
高スケーラビリティ

デメリット
パケット転送性能

◻ 先行研究†によるI/O性能の向上

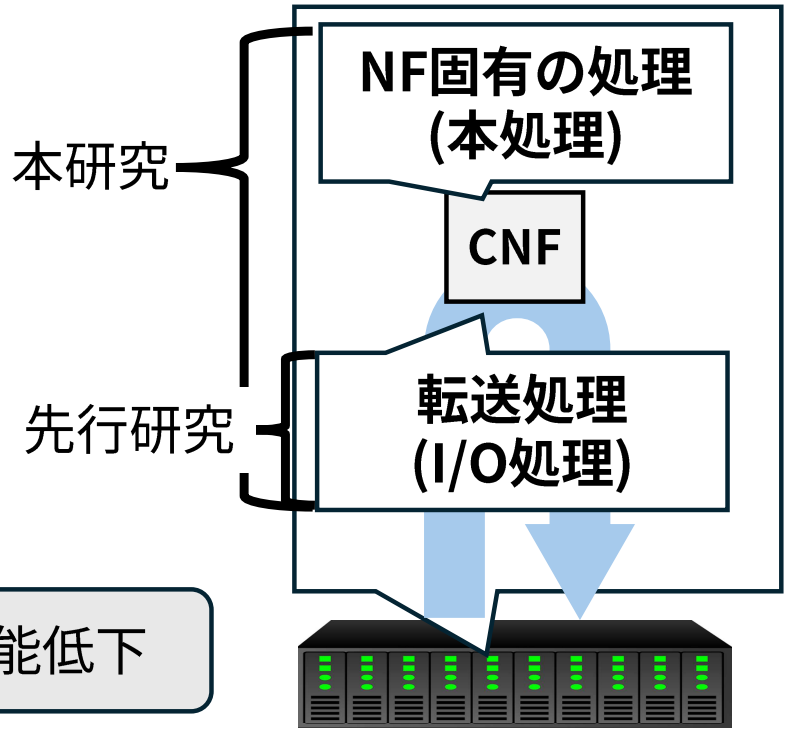
- 20Mpps(DPDK) ▶ 130Mpps
- 転送処理のみに着目

◻ 本処理の高速化

- ヘッダ解析・更新
- テーブルマッチング処理

No.	key	action
1	dst_ip=○	action A
2	src_mac=△ src_port=□	action B
⋮		
n	dst_ip=● dst_port=■	action N

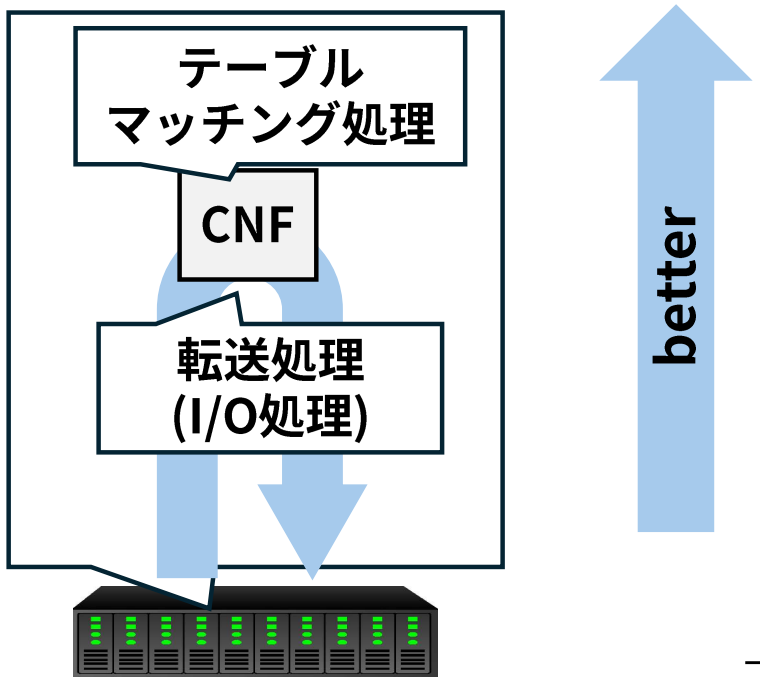
処理に伴う性能低下

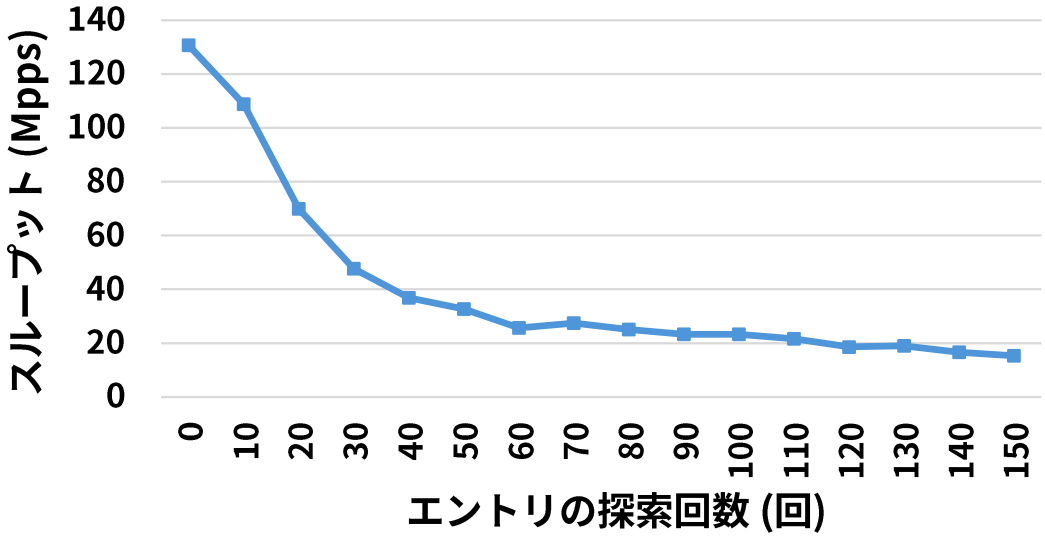


→ **テーブルマッチング処理の評価**

† A. Yamada et al., “Rethinking Message Buffer Structures for 100 Mpps Cloud-native Network Functions”, IEEE FNWF 2024

テーブルマッチング処理の性能



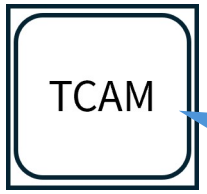


エントリの探索回数	0	50	100	150
L1dキャッシュヒット率	99.99%	99.99%	99.99%	99.99%

ソフトウェア実装の限界

結果

- **20エントリ以降、性能が半分以下**
- **キャッシュヒット率の維持**



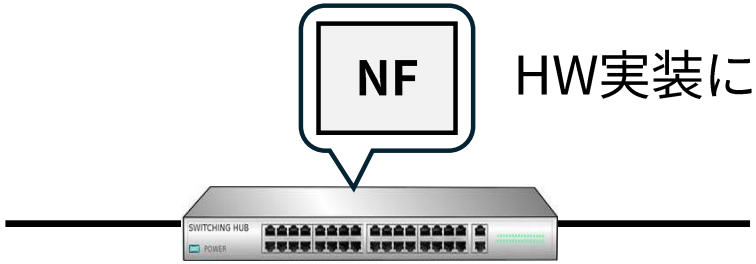
高速なテーブルマッチング処理

全エントリの同時並列マッチング

HWとの協調による高速化が必要

WBSの活用

WBSのみの構成



HW実装による高速化

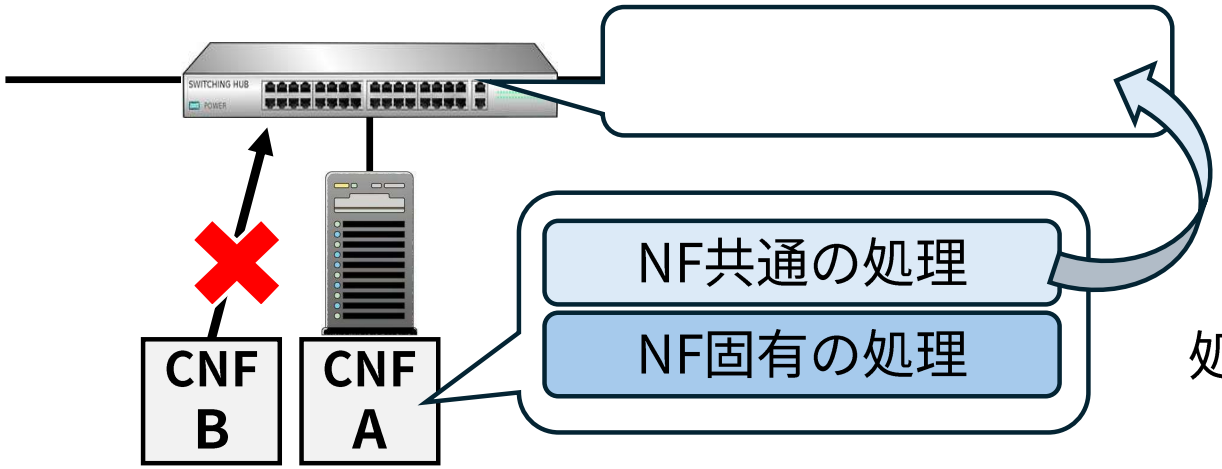
Whitebox Switch

問題

柔軟なNF配置が困難

CNFとしての実装形態を維持

WBS専有の構成



問題

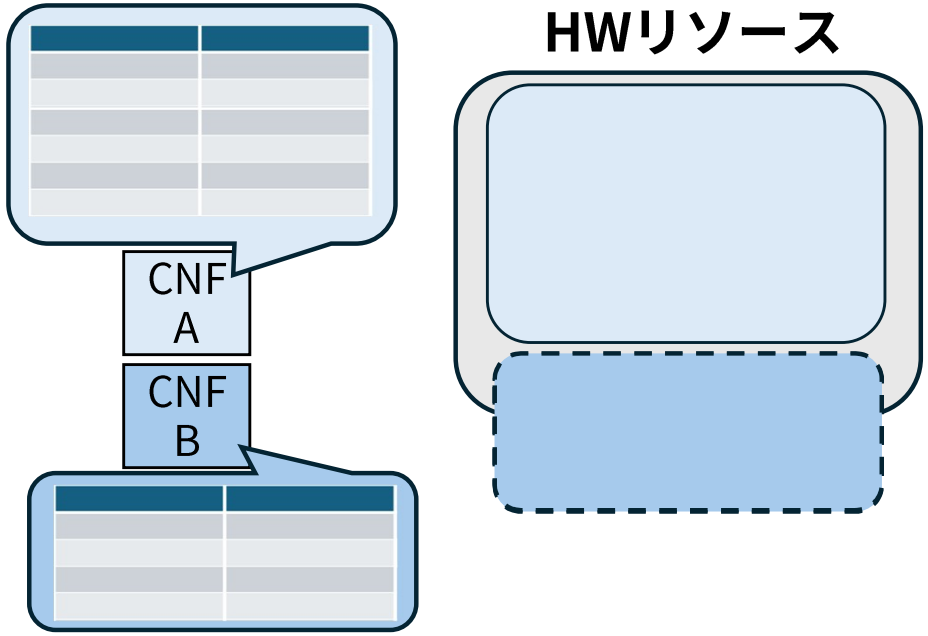
マルチテナンシーの欠如

処理のオフロードによる高速化

異なるNF間でWBSを共有する仕組みが必要

NF間でのWBSの共有

静的な共有

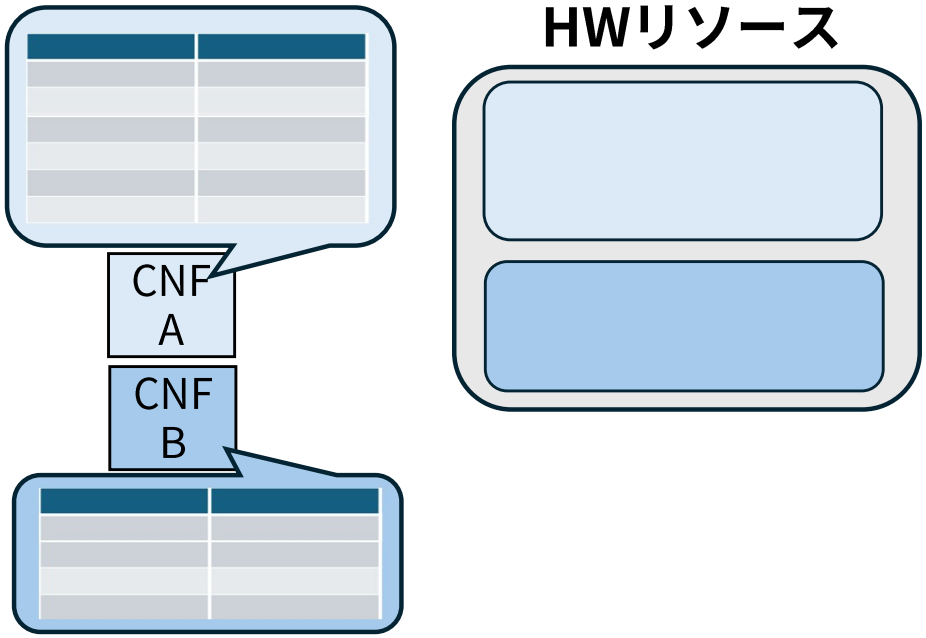


常に最大性能で検索可能

HWリソースの浪費

HWリソースの枯渇

動的な共有



HWリソースの有効活用

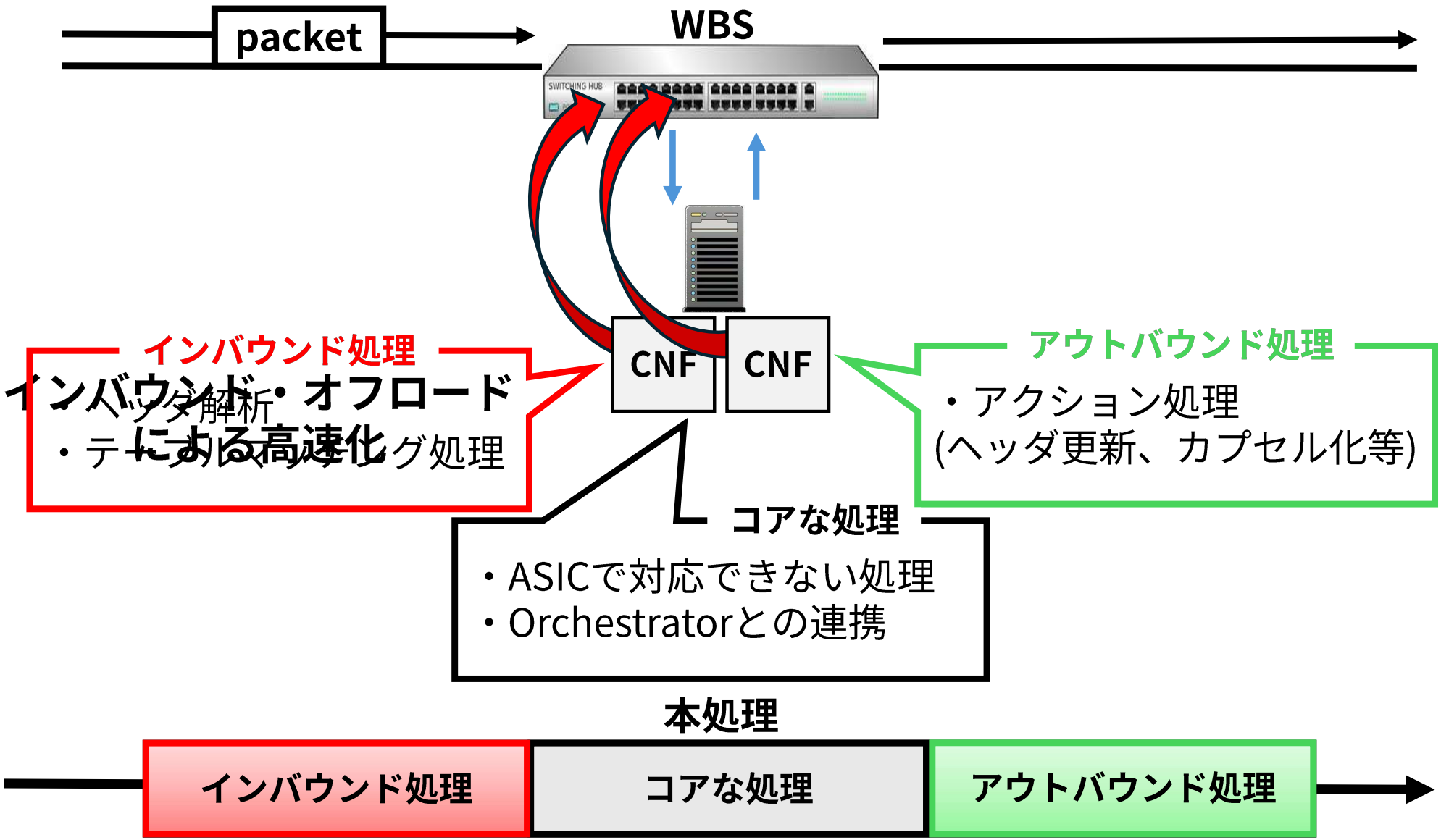
制御負荷の増大

柔軟なリソース割り当てが可能

CNFとWBSの“動的”な共有が必要

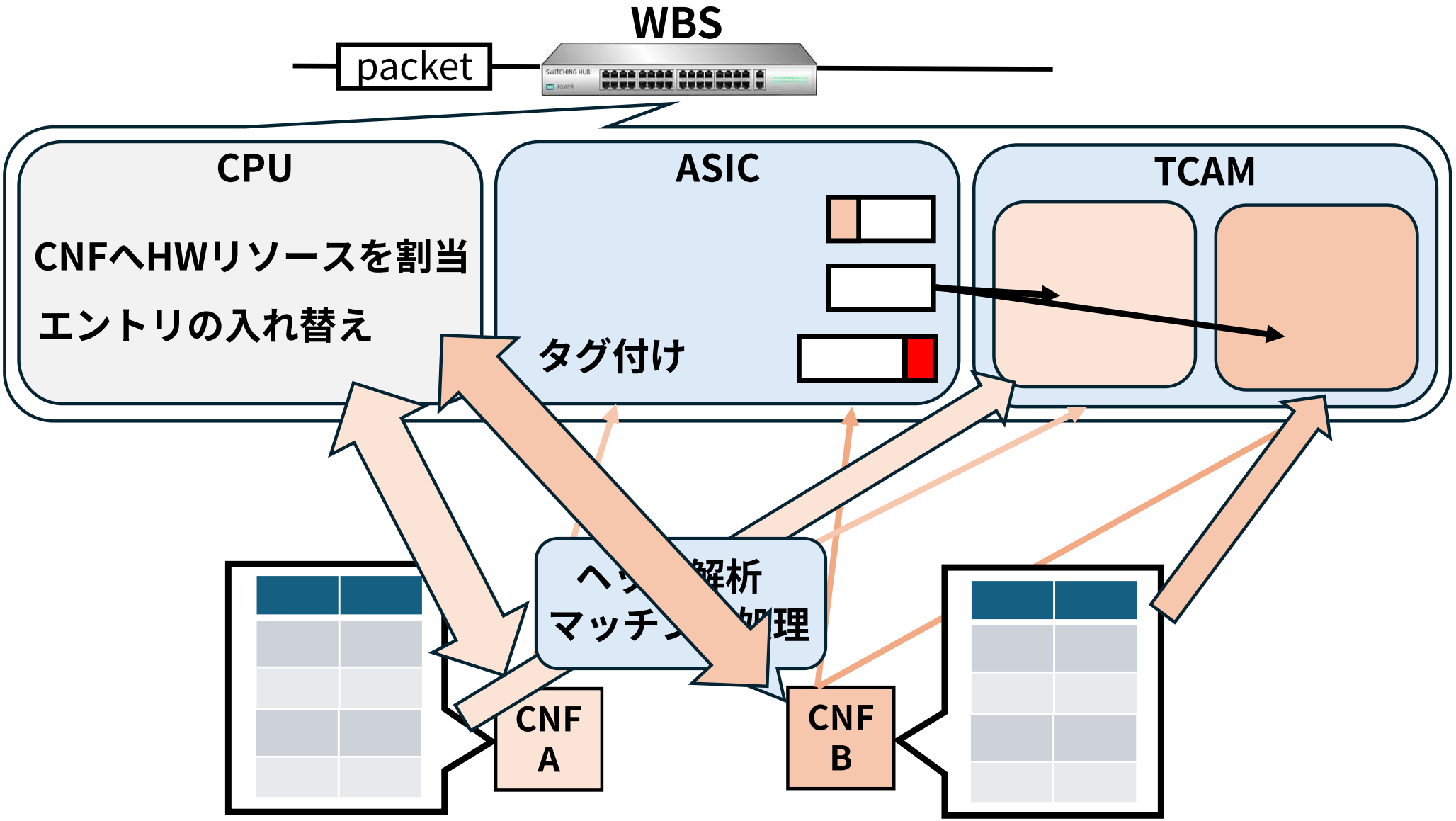
提案アーキテクチャの設計

ロCNFが処理の一部をWBSへオフロード



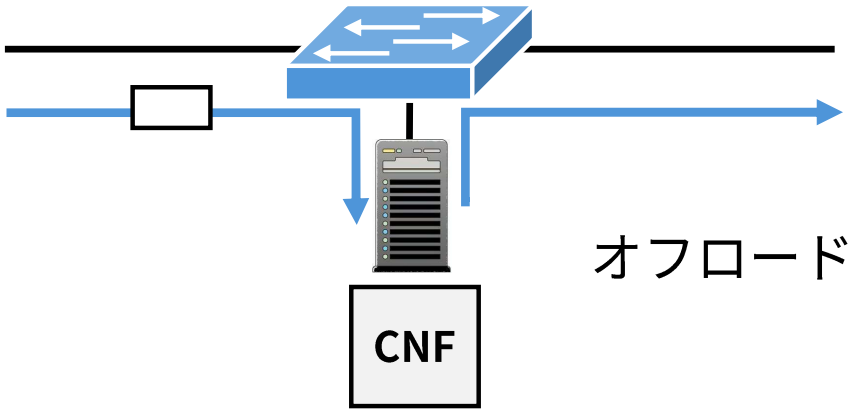
インバウンド・オフロード

- CNFがHW (ASIC, TCAM) を共有リソースとして扱う

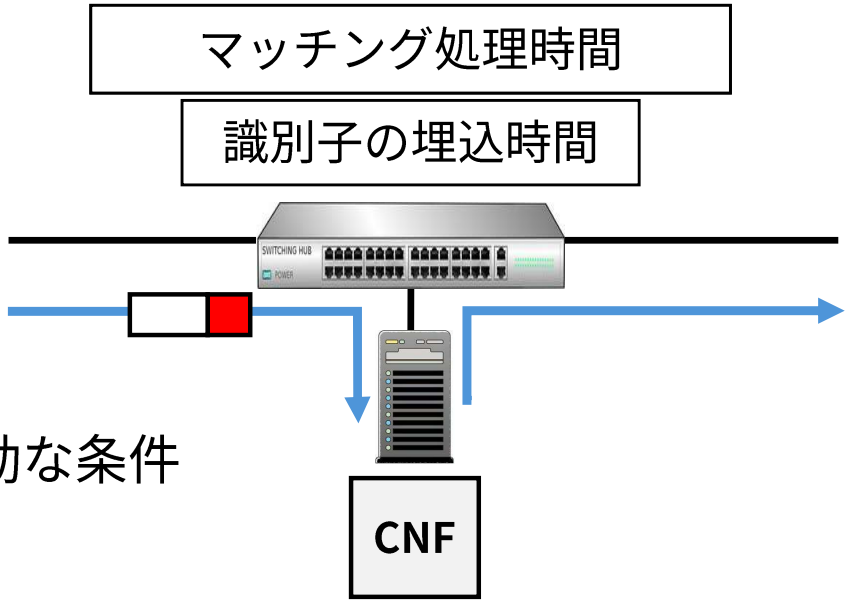


オフロードの有効性

従来アーキテクチャ



提案アーキテクチャ



>

オフロードが有効な条件

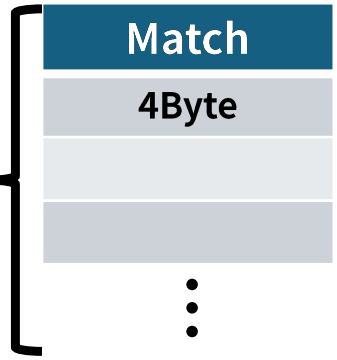
マッチング処理時間 ← 本研究が行った計測

CNFの評価環境

CPU	Core i9 11900K
Clock	3.5 GHz
Memory Clock	3200 MHz

テーブルの構造

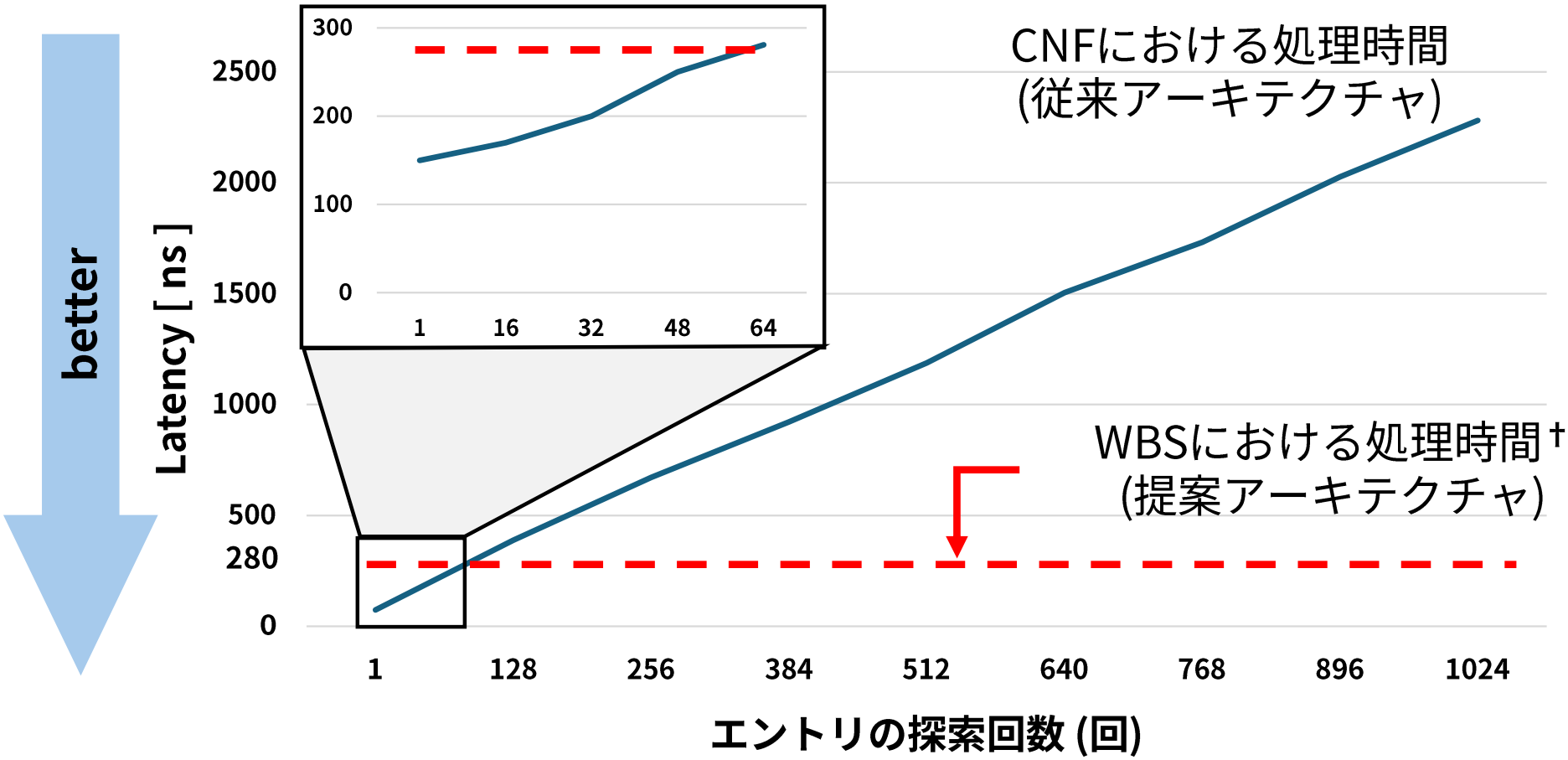
エントリ数
1024



WBSの評価環境

機種名	APS BF2556X-1T
ASIC	Intel Tofino

結果



結果

- **64エントリ以降、提案手法の方が有利である**

→ **優先度の高い64エントリをCNFに残すことで高速化を図る**

† David Franco, et al., "A comprehensive latency profiling study of the Tofino P4 programmable ASIC-based hardware," *Computer Communications*, March 2024.

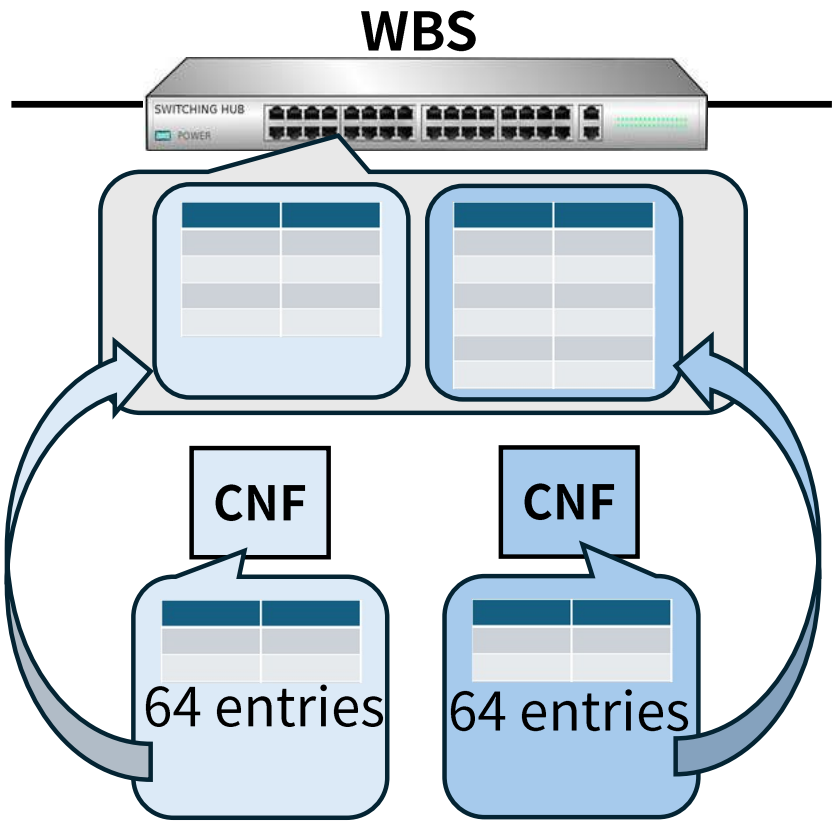
まとめと今後の課題

◻ 課題

- CNFのテーブルマッチング処理に伴う性能低下

◻ 提案

- CNF主体によるWBSとの動的な協調アーキテクチャ

◻ 今後の展望

- 高頻度な64エントリのCNF配置による処理の高速化を検討
- CNF-WBS間でのメッセージ体系の設計